

Sign in with Apple – Backend Integration Guide

App Name: JOL Puzzles

Prepared For: Backend (Django) Team

Prepared By: Mobile Application Developer

Date: April 12, 2026

Objective

To implement **Sign in with Apple** authentication for the JOL Puzzles mobile application. This integration ensures compliance with Apple's App Store Review Guideline 4.8 and enables secure user authentication.

Apple Developer Credentials

The following credentials have been configured and are required for backend token verification:

Credential	Value
App Name	JO Puzzles
Bundle ID (App ID)	<code>com.jacklookman.jolpuzzles</code>
Team ID	<code>B5U47R6W84</code>
Key ID	<code>6G58A6U47M</code>
Private Key File	<code>AuthKey_6G58A6U47M.p8</code> <i>(shared securely)</i>
Enabled Service	Sign in with Apple
Apple Developer Entity	Jack Lookman Limited
Account Type	Organization

 **Important:** The `.p8` private key is confidential and must be stored securely. It cannot be downloaded again from Apple.

Configuration Confirmation

The Apple Sign-In key has been successfully configured with the following details:

- **Key Name:** JO Puzzles
- **Key ID:** 6G58A6U47M
- **Team ID:** B5U47R6W84
- **Capability Enabled:** Sign in with Apple
- **App Association:** `com.jacklookman.jolpuzzles`

No Service ID is required since authentication is performed through a native iOS application.

Authentication Flow

1. The user taps **Sign in with Apple** in the mobile app.
 2. Apple authenticates the user.
 3. Apple returns an **identity token** and **authorization code** to the app.
 4. The mobile app sends this data to the backend.
 5. The backend verifies the token with Apple.
 6. The backend logs in or registers the user.
 7. The backend returns a session token (JWT) to the mobile app.
-

Backend API Specification

Endpoint

```
POST /api/auth/apple/
```

Request Body

```
{
  "identity_token": "APPLE_IDENTITY_TOKEN",
  "authorization_code": "AUTHORIZATION_CODE",
  "first_name": "John",
  "last_name": "Doe",
  "email": "john@example.com"
}
```

Expected Response

```
{
  "success": true,
  "token": "JWT_TOKEN",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

Token Verification Requirements

The backend must validate the Apple identity token using Apple's public keys.

Apple Public Keys Endpoint

```
https://appleid.apple.com/auth/keys
```

Verify the Following Claims

Claim	Expected Value
<code>iss</code>	https://appleid.apple.com
<code>aud</code>	com.jacklookman.jolpuzzles

Claim	Expected Value
<code>exp</code>	Must be valid and not expired
<code>sub</code>	Unique Apple User Identifier
<code>email</code>	User's email (may be private relay)

Recommended Python Libraries

Install the required dependencies:

```
pip install python-jose requests cryptography
```

Django Implementation Example

Sample Verification Logic

```
import requests
from jose import jwt

APPLE_PUBLIC_KEYS_URL="https://appleid.apple.com/auth/keys"
TEAM_ID="B5U47R6W84"
AUDIENCE="com.jacklookman.jolpuzzles"

def verify_apple_identity_token(identity_token):
    # Fetch Apple public keys
    response=requests.get(APPLE_PUBLIC_KEYS_URL)
    apple_keys=response.json()["keys"]

    # Get token header
    header=jwt.get_unverified_header(identity_token)

    # Find matching key
    key=next(
        (k for k in apple_keys if k["kid"]==header["kid"]),
        None
```

```

    )

    if not key:
        raise Exception("Invalid Apple public key")

    # Verify token
    payload = jwt.decode(
        identity_token,
        key,
        algorithms=["RS256"],
        audience=AUDIENCE,
        issuer="https://appleid.apple.com"
    )

    return payload

```

User Management Logic

Scenario	Action
New User	Create a new account using Apple credentials
Existing User	Log in the user
Hidden Email	Store Apple relay email securely
Missing Name	Save name only during first login
Unique Identifier	Store the <code>sub</code> as <code>apple_user_id</code>

Data Mapping

Apple Field	Database Field
<code>sub</code>	<code>apple_user_id</code>
<code>email</code>	<code>email</code>
<code>givenName</code>	<code>first_name</code>
<code>familyName</code>	<code>last_name</code>

Apple Field	Database Field
<code>identity_token</code>	<code>token_reference</code>

Testing Checklist

Test Case	Status
Token verification successful	☐
New user registration	☐
Existing user login	☐
Private Relay email handling	☐
JWT generation	☐
Invalid token handling	☐
Expired token handling	☐

Security Best Practices

- Store the `.p8` key securely in environment variables.
- Never commit the key to version control.
- Use encrypted secrets management tools.
- Always validate token signatures and claims.
- Enforce HTTPS for all authentication endpoints.

Environment Variables

```
APPLE_TEAM_ID=B5U47R6W84
APPLE_KEY_ID=6G58A6U47M
APPLE_CLIENT_ID=com.jacklookman.jolpuzzles
APPLE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END
PRIVATE KEY-----"
```

Final Deliverables

Deliverable	Owner
Apple Token Verification API	Backend Team
Secure Storage of Credentials	Backend Team
Apple Sign-In Integration	Mobile Developer
Testing & Validation	Both Teams
Production Deployment	Backend Team



Contact Information

If any clarification is required, please contact the mobile development team for coordination and testing.



Definition of Done

-  Apple identity tokens are verified securely.
-  Users can log in and register using Apple.
-  The authentication flow works seamlessly with the mobile app.
-  The integration complies with Apple's App Store guidelines.
-  The app is ready for resubmission to the App Store.